Máster en Inteligencia Artificial · UTAMED
Deep Learning · Ruta de Aprendizaje Autónomo · Parte 1

Del problema al modelo: clasificación de imágenes para picking robotizado en almacén

Autor: Víctor Ferrer Sector: Logística — automatización de picking

Tarea 1 · Análisis del problema y selección de arquitectura (RAAP)

1.1 Definición del problema

El cliente es una empresa de logística que está automatizando uno de sus almacenes con robots móviles de picking. Cada robot tiene que recoger los productos de las estanterías y llevarlos a la sección que toca, así que necesita reconocer qué tipo de producto tiene delante a partir de la imagen que captura su cámara. Hoy esa clasificación la hacen los operarios a mano, lo que ralentiza el flujo y se convierte en un cuello de botella en horas punta. El objetivo es claro: automatizar la decisión con un modelo de visión por computador para liberar a los operarios y dedicarlos a tareas con más valor añadido.

Visto así, el problema encaja con una clasificación multiclase de imágenes. A partir de una imagen, el modelo predice a cuál de las 10 categorías del sistema de almacenamiento pertenece (vehículos, animales, etc.). Es la formulación más natural por tres motivos: las clases están bien definidas y no se solapan, las imágenes son naturales (no son señales ni texto), y existen datasets etiquetados para entrenar de forma supervisada.

1.2 Selección de arquitectura

La propuesta es usar una red neuronal convolucional (CNN), en concreto una ResNet18 preentrenada en ImageNet a la que se le aplica fine-tuning. Hay varios motivos detrás de esta elección. Primero, las CNN siguen siendo el caballo de batalla para clasificar imágenes naturales: aprovechan la localidad espacial, son equivariantes ante traslaciones y comparten parámetros, lo que ayuda mucho cuando el dataset no es enorme. Segundo, las skip-connections de ResNet permiten entrenar redes profundas sin que el gradiente se desvanezca por el camino. Tercero, ResNet18 ronda los 11 millones de parámetros, una cifra manejable con una GPU modesta o incluso con CPU, y desplegable en el ordenador embarcado del propio robot. Por último, los pesos preentrenados en ImageNet ya capturan filtros de bordes, texturas y formas que sirven directamente para nuestras 10 categorías.

¿Por qué no otras alternativas? Los Vision Transformers piden mucho más volumen de datos cuando se entrenan desde cero, y sus versiones preentrenadas pesan más en inferencia. Los

modelos generativos (VAE, GAN, Diffusion) sirven para sintetizar muestras, no para clasificar. Las RNN/LSTM y los Transformers de NLP funcionan con datos secuenciales, no con imágenes. Y los algoritmos de RL como DQN o PPO se aplican en problemas de decisión secuencial; el robot podría usarlos a más alto nivel para planificar rutas, pero no son la herramienta para esta tarea.

1.3 Análisis de los datos

Para el prototipo se trabaja con CIFAR-10, uno de los datasets sugeridos en el enunciado. Contiene 60.000 imágenes RGB de 32×32 píxeles, repartidas a partes iguales en 10 clases (avión, coche, pájaro, gato, ciervo, perro, rana, caballo, barco y camión). Es un dataset balanceado, etiquetado, de acceso público y se descarga directamente desde torchvision en una sola línea de código. El reparto estándar viene con 50.000 imágenes para train y 10.000 para test; aquí se reorganiza reservando 5.000 del train como conjunto de validación.

CIFAR-10 actúa como dataset proxy para validar el pipeline completo de extremo a extremo. En un proyecto real, antes de cualquier despliegue habría que sustituirlo por imágenes capturadas con la propia cámara del robot. Lo razonable serían entre 1.000 y 5.000 imágenes etiquetadas por categoría real de producto, recogidas en distintas condiciones de iluminación, ángulos y embalajes. Para amortiguar la escasez inicial de datos del cliente se combinan tres palancas: transfer learning (implícito al partir de ResNet18 preentrenada), data augmentation (flips, recortes aleatorios, color jitter) y, si llegara a hacer falta, generación de imágenes sintéticas.

1.4 Riesgos técnicos y mitigación

Los principales riesgos del entrenamiento y del despliegue, junto con la forma de mitigarlos, se resumen en la tabla siguiente:

Riesgo técnico	**Estrategia de mitigación**
Overfitting	Data augmentation (flips, crops, color jitter), dropout en la cabeza, weight decay y early stopping sobre la accuracy de validación.
Vanishing gradients	Skip-connections de ResNet, BatchNorm en los bloques convolucionales, inicialización Kaiming y optimizador Adam.
Confusión entre clases similares	Análisis de la matriz de confusión, augmentation específica para las clases peor clasificadas y, si procede, ampliación del dataset.
Drift en producción	Monitorización de la accuracy y de la distribución de probabilidades; reentrenamiento periódico con casos nuevos o dudosos (active learning).

Riesgo técnico	Estrategia de mitigación
Diferencia entre dataset y realidad	Validación con imágenes reales del almacén antes del despliegue y fase de shadow deployment de 2–3 semanas.

Tarea 2 · Diseño técnico de la solución (RAAR)

2.1 Diseño de la arquitectura

La arquitectura sigue un pipeline secuencial: preprocesado → backbone preentrenado → cabeza de clasificación. La regularización viene por dos vías: la BatchNorm interna de los bloques residuales y un dropout aplicado en la cabeza. Las activaciones son ReLU y la salida pasa por una softmax para obtener la probabilidad de cada una de las 10 clases.

Bloque	Componentes	Salida
Entrada	Imagen 64×64 RGB normalizada (media/std de ImageNet)	3×64×64
Backbone ResNet18 (congelado)	Conv7×7 + 4 stages residuales (BatchNorm, ReLU, skip)	512×2×2
Pooling	Adaptive Average Pooling 2D	512
Cabeza de clasificación	Linear(512 → 10)	10 (logits)
Salida	Softmax → probabilidades por clase	10 prob.

En la primera fase se congela todo el backbone y se entrena solamente la cabeza, que tiene unos 5.000 parámetros. Esto reduce mucho el coste computacional y limita el riesgo de overfitting cuando los datos son escasos. Más adelante, ya con datos del cliente, se descongelarían los dos últimos stages y se haría fine-tuning gradual con un learning rate diez veces menor.

2.2 Pipeline de entrenamiento

Los hiperparámetros principales son: optimizador Adam con learning rate 1e-3 (aplicado solo sobre la cabeza), CrossEntropyLoss como función de pérdida (estándar en clasificación multiclase, admite el parámetro weight para compensar desbalances), batch de 128 y 5 épocas, suficientes en este régimen porque el grueso del aprendizaje ya viene de los pesos preentrenados. Las semillas de Python, NumPy y PyTorch se fijan a 42 para que los experimentos sean reproducibles. Durante el entrenamiento se logean tanto la loss como la accuracy de train y validación, y se guarda el mejor checkpoint según la accuracy de validación. Si se ejecutara en GPU, se activaría además mixed precision, que acelera aproximadamente 2× sin pérdida apreciable de precisión.

2.3 Métricas de evaluación

Como CIFAR-10 está perfectamente balanceado, la accuracy es una métrica representativa y se utiliza como referencia principal. Para complementarla se reporta el F1 macro, que evita que las clases más fáciles dominen el promedio, y la precision/recall/F1 desglosado por clase, que es lo que permite ver qué pares de clases confunde más el modelo (lo habitual es que perro/gato y ciervo/caballo den más problemas que el resto). La matriz de confusión es el artefacto que mejor funciona para hablar con el cliente: traduce los errores a casos concretos y deja claro qué clases hay que priorizar en una segunda iteración. De forma opcional, la accuracy top-1 se acompaña de la top-3, útil cuando interesa medir si el modelo "está cerca" aunque no acierte exactamente.

2.4 Plan de validación

La validación interna se plantea en tres niveles. Primero, un split estratificado train/val/test: 5.000 imágenes del train de CIFAR-10 quedan como validación y el test set oficial de 10.000 imágenes se mantiene intacto para el reporte final. Segundo, una cross-validation con k=5 sobre train+val para estimar la varianza y elegir hiperparámetros, reportando media y desviación típica de la accuracy. Tercero, un test set blindado que no participa en ninguna decisión de modelado.

Antes del despliegue en producción se prevé una fase de validación externa. Durante 2–3 semanas se haría un shadow deployment en una zona del almacén, donde el modelo predice pero no actúa; los operarios validan en paralelo y sus etiquetas se comparan con las predicciones del modelo. A continuación, un A/B testing entre una zona con modelo y otra sin él, midiendo KPIs de negocio: productos por hora, errores de clasificación y tiempo medio por pieza. El criterio de aceptación se fija en accuracy ≥ 0,80 en el test de CIFAR-10 (un objetivo razonable para ResNet18 fine-tuned) y ≥ 0,95 sobre datos reales del almacén una vez reentrenado.

Referencias

Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press.

Prince, S. J. D. (2023). Understanding Deep Learning. MIT Press.

He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR.

Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images (CIFAR-10).

Chollet, F. (2021). Deep Learning with Python (2ª ed.). Manning Publications.